Inside the vLLM Scheduler

How V1 decides what to run, manages the KV cache, and mixes prefill with decode

A tutorial for researchers and infrastructure engineers

Preface

These notes explain how the vLLM V1 engine's scheduler works: how it picks which requests to run on each forward pass, how it manages the KV cache that LLM inference fundamentally depends on, how prefill and decode coexist in a single batch, and what happens when memory pressure forces eviction. The intent is to give a reader who has used vLLM as a black box enough mechanical understanding to predict its behavior under unusual workloads, tune its parameters, and reason about extensions.

The exposition is grounded in V1's behavior, which is the supported branch as of this writing. Where V0 differs (notably swap preemption, which V1 dropped), the distinction is noted. The focus is on the scheduling and memory-management layer, not the model executor or distributed coordination.

The reader is assumed to know what an LLM is, what prompts and tokens are, and the general distinction between prefill (processing the prompt) and decode (generating one output token at a time). Familiarity with continuous batching helps but is reviewed briefly.

1. Why Continuous Batching Exists

A single GPU running an LLM forward pass at batch size 1 is memory-bandwidth bound. Loading the model weights from HBM dominates iteration time; the actual compute occupies a small fraction of the available FLOPS. Adding more sequences to the batch is nearly free per-iteration because the bandwidth would be spent loading those weights regardless. This is the central economic fact of LLM inference: throughput scales almost linearly with batch size until compute itself becomes the bottleneck.

Naively batching N requests by waiting for all N prompts to arrive, then processing them together to completion, leaves enormous performance on the table. Requests that finish early sit idle while the slowest one completes. Newly arriving requests wait until the next batch boundary. The GPU spends substantial fractions of its time at lower batch sizes than it could otherwise sustain.

Continuous batching fixes this by reorganizing the work. Instead of one batch processing one set of requests to completion, the engine runs in iteration-level steps where the batch composition can change every forward pass. A request that just finished is removed; a request that just arrived can join. The batch contains both decoding (one-token-at-a-time) and prefill (many-tokens-at-once) work simultaneously. Each forward pass is a fresh decision about what to process.

This shifts the problem from "how do I batch?" to "how do I schedule?" The scheduler — the focus of these notes — must decide, every iteration, *which requests get to advance*, *by how many tokens*, and *what to do when KV memory runs out*. The answers determine throughput, latency, fairness, and stability under load.

2. Engine Architecture

The vLLM LLM Engine wraps several components into a unified request-response interface. Understanding the layout matters because the scheduler is one component among several, and the boundaries determine what it does and doesn't control.

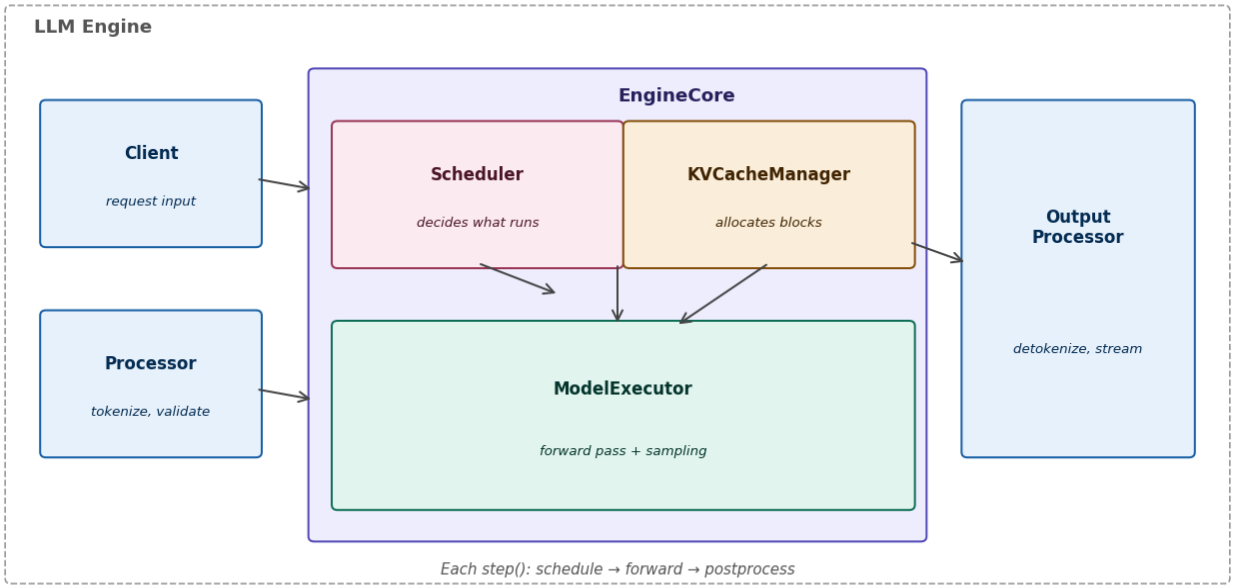

Figure 1. The LLM Engine's internal components. The Client receives requests; the Processor tokenizes and validates them; the EngineCore (Scheduler + KVCacheManager + ModelExecutor) does the actual scheduling and inference work; the Output Processor detokenizes results and streams them back. Each invocation of step() runs the schedule → forward → postprocess pipeline once.

Client. Receives requests from external callers, attaches metadata (sampling parameters, lora adapters, stop conditions), forwards them to the Processor. In production this is the API server.

Processor. Tokenizes the prompt, validates token IDs against the vocabulary, applies any input transforms, and hands a Request object to the EngineCore. Tokenization is intentionally outside the EngineCore's critical path so it doesn't block the model forward.

EngineCore. The heart of vLLM. Contains the Scheduler (decides what to run), the KVCacheManager (decides where to store it), and the ModelExecutor (runs the forward pass and samples). All scheduling decisions and KV memory management happen here.

Output Processor. Detokenizes the sampled token IDs back to strings, checks stop conditions, streams partial outputs to the Client. Cleans up state when a request finishes.

The engine is driven by a `step()` method that runs three stages in sequence. First, **schedule** picks the requests for this iteration and reserves their KV cache slots. Second, **forward pass** runs the model on the chosen batch and samples one token per active sequence. Third, **postprocess** appends sampled tokens to their requests, detokenizes, checks stop conditions, and removes completed requests. The cycle repeats indefinitely until the request queue empties.

3. The KV Cache: Paged, Block-Based Memory

Every transformer layer maintains a cache of keys and values produced by previous tokens. Decoding the next token requires reading these K/V vectors for every prior token in the context. A 7B model with 4K context per request and a typical attention head layout stores tens of MB of K/V state per request; with hundreds of concurrent requests, the KV cache becomes the dominant memory consumer on the GPU.

vLLM's PagedAttention design treats the KV cache the way an operating system treats RAM: as a pool of fixed-size blocks that can be allocated to requests on demand, freed when requests complete, and tracked through indirection. A request does not get a contiguous region of memory; it gets a list of blocks scattered through the pool. This indirection costs a small amount of attention-kernel overhead and buys enormous memory utilization.

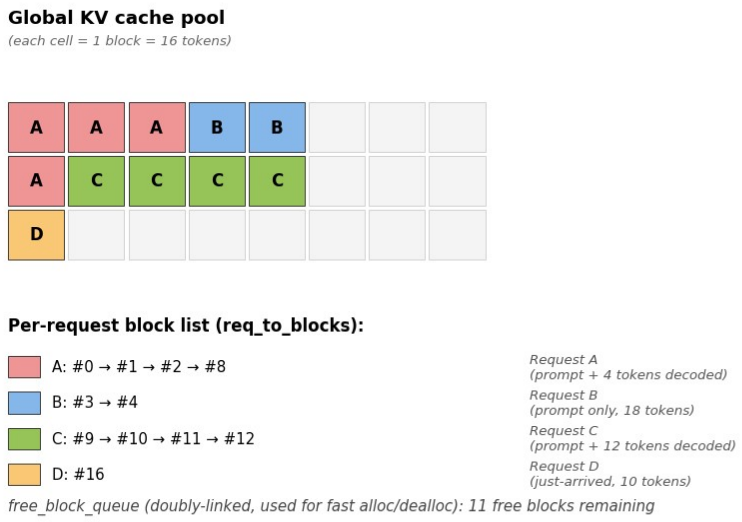

Figure 2. KV cache memory layout. The global pool is a grid of fixed-size blocks (default 16 tokens per block). Each request owns a list of blocks; the blocks need not be contiguous. The free_block_queue maintains all unallocated blocks for fast O(1) allocation.

3.1 Block size and contents

Default `block_size = 16` tokens. A block holds the K and V vectors for that many tokens across all attention heads of one layer. For a non-MLA model the per-block byte size is:

```
block_bytes = 2                    # K and V
            × block_size           # 16 tokens
            × num_kv_heads         # KV heads per layer
            × head_size            # dimension per head
            × dtype_bytes          # 2 for bf16, 4 for fp32
```

For a typical 7B model with 32 KV heads, 128 head_size, bf16 storage, one block is 2 × 16 × 32 × 128 × 2 = 262 KB. A typical deployment allocates 10-100 GB of GPU memory to the KV pool, yielding tens to hundreds of thousands of free blocks at start.

3.2 Per-request block list

Each running request has an entry in `req_to_blocks`: a list of block IDs, in token order. When the request needs to store K/V for new tokens, the scheduler appends blocks. When the request finishes (or is evicted), its blocks are released back to the free queue. The list order is preserved because attention kernels need to read K/V in token order; the underlying physical blocks can be anywhere in the pool.

3.3 The free block queue

Unallocated blocks live in `free_block_queue`, a doubly-linked list. Allocation pops from one end (O(1)); deallocation pushes (O(1)). When prefix caching is enabled (Section 8), recently-freed blocks may be retained for reuse rather than immediately recycled — they get added back to the head of the queue and reclaimed only when nothing else is free.

3.4 Allocation: the central operation

The function `allocate_slots(request, new_tokens)` is called every time a request needs to write more K/V state. It performs three steps:

1. Compute the number of new blocks needed: $\lceil new\_tokens \,/\, block\_size \rceil$, adjusted for partial fill of the request's current last block.

2. Check whether the free queue has enough blocks. If yes, pop them, append to the request's block list, return success. If no, return failure (which triggers preemption in the scheduler — see Section 7).

3. On a successful path with prefix caching enabled, check whether any of the needed blocks correspond to a previously-seen prefix; if so, point at the cached blocks rather than allocating new ones.

This function is the single most important gate in the engine. Every request's progress passes through it. The scheduler's decision logic is largely structured around predicting and reacting to `allocate_slots` outcomes.

4. The Scheduler: How Decisions Are Made

The scheduler holds two queues. The waiting queue contains requests that have arrived but not yet entered the batch (typically because they're new). The running queue contains requests that are mid-execution — either in prefill (still processing their prompt) or in decode (producing output tokens).

Every `step()` invocation, the scheduler is given:

- the current waiting and running queues
- a per-iteration `token_budget` — the maximum number of tokens that can be processed this forward pass (typically equal to `max_num_batched_tokens`, default 8192 or similar)
- the current state of the KV cache (free block count, allocations)

It returns a set of scheduled requests and the per-request token counts that will be processed. The job is to maximize useful work within the token budget without exceeding KV capacity.

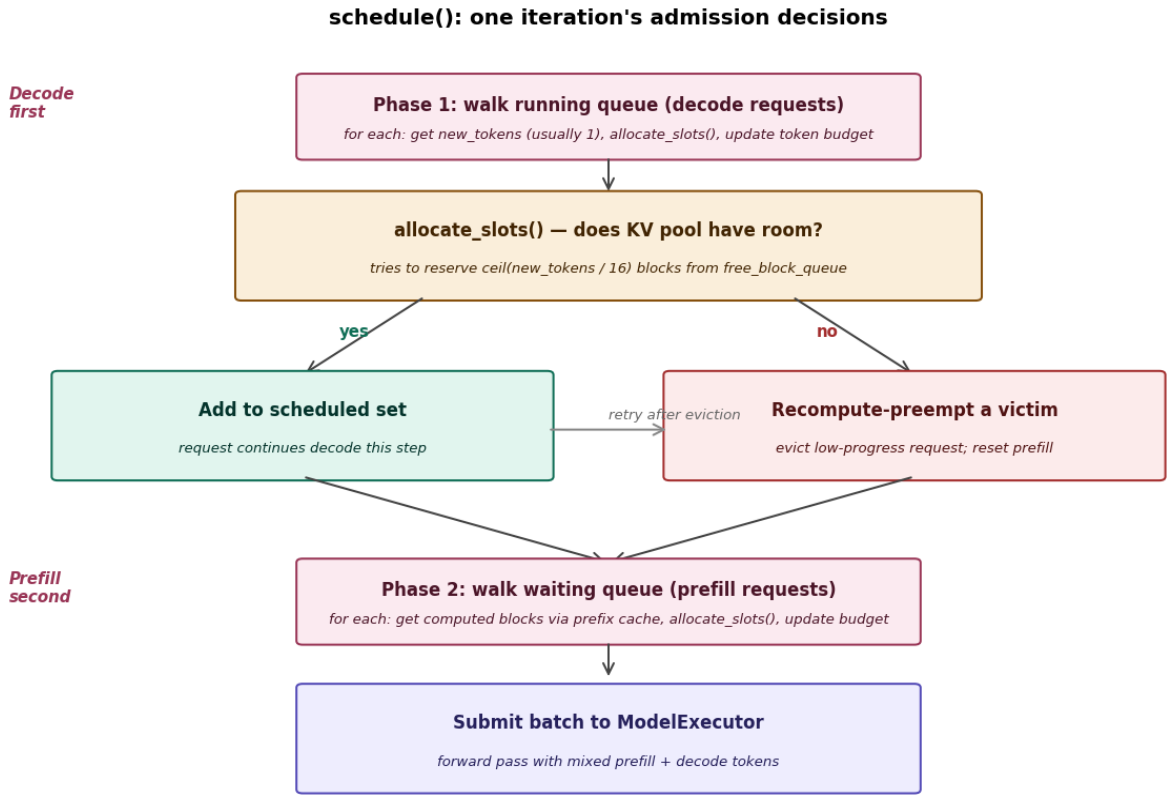

Figure 3. The schedule() decision flow. Two phases: first walk the running queue (decode requests already in flight); for each, call allocate_slots and either add to the scheduled set or trigger recompute preemption. Second, walk the waiting queue (prefill requests) in order, again calling allocate_slots. The two phases share the token budget; decode consumes from it first.

4.1 Phase 1: serve in-flight decode requests

Decode requests get priority. The reasoning: they've already done expensive prefill work and have an established KV cache footprint. Letting them finish quickly frees those blocks and reduces overall queue depth. Starving them in favor of fresh prefills would waste sunk cost and inflate latency for users who are already waiting on responses.

For each request in the running queue:

1. Compute the number of new tokens to process. For a vanilla decode this is 1 (one new output token). With speculative decoding it's k+1 (k draft tokens plus the verification step). With asynchronous scheduling it may be slightly more.

2. Call `allocate_slots(request, new_tokens)`. This may need a new block (when the current last block fills up) or may not (when the current block has room).

3. If allocation succeeds, add the request to the scheduled set and debit `new_tokens` from the token budget. If allocation fails, trigger preemption (Section 7) and try again.

Continue until the running queue is exhausted or the token budget runs out.

4.2 Phase 2: admit new prefill requests

With remaining token budget, the scheduler walks the waiting queue in FIFO order. For each request:

1. Determine how many prompt tokens to prefill this step. Without chunked prefill, this is the entire prompt — but capped at `token_budget` remaining. With chunked prefill (Section 6), capped at `long_prefill_token_threshold` or budget remaining, whichever is smaller.

2. If prefix caching is enabled, call `find_longest_cache_hit(prompt)` to find a previously-computed prefix. Skip prefill work for those tokens and reuse the cached blocks.

3. Call `allocate_slots` for the non-cached portion of the prefill.

4. On success, move the request from waiting to running and debit the budget. On failure (and the waiting queue is non-empty), stop scheduling; the engine will revisit on the next iteration.

Phase 2 stops when either the waiting queue is empty, the token budget is exhausted, or allocation fails.

4.3 Token budget arithmetic

The token budget is the single most important parameter for tuning latency-throughput trade-offs. A small budget (say 1024) caps the amount of compute per iteration, keeping iteration time low and prefills small — good for inter-token latency on decode-heavy workloads. A large budget (say 16384) lets the engine pack big prefills into single iterations, boosting throughput at the cost of longer iterations and higher latency for tokens emitted in those iterations.

The default value (typically `max_num_batched_tokens` = 8192) is a compromise. Production deployments often tune this based on the workload. A real-time chat service might set it low; a batch summarization job might set it high.

5. Mixing Prefill and Decode in One Step

V1's most consequential design change over V0 is that a single forward pass can process both prefill and decode tokens. V0 had separate prefill-only and decode-only steps, alternating between them. V1 merges them: a single batched forward sees a flat sequence of mixed prefill and decode tokens with metadata distinguishing which positions belong to which request.

Inside the kernel, the engine flattens all sequences into a single "super-sequence." If three requests are scheduled with token counts 1 (decode), 1 (decode), and 64 (prefill chunk), the input is a flat tensor of 1 + 1 + 64 = 66 tokens. Attention is computed with per-token position indices and per-request causal masks so that each sequence only attends to its own preceding tokens. The result is sliced back per-request before sampling.

5.1 Why mixing matters

Three reasons V1's choice is significant:

- **No idle decode tokens during prefill bursts.** If a prefill arrives mid-stream, V0 would block all decodes until the prefill completed (or it would run the prefill alone, blocking decodes). V1 includes them in the same forward, so decode latency for in-flight requests is preserved.

- **Better GPU utilization.** Prefills produce many tokens per pass; decodes produce one. Mixing them packs more useful work into each forward, keeping the batch closer to peak throughput.
- **Cleaner scheduling logic.** The scheduler doesn't need to alternate between modes or buffer requests of one type while serving the other. A single token-budget allocation per step covers everything.

5.2 The cost of mixing

Mixing isn't free. Two non-obvious costs:

Attention kernel complexity. A pure-decode batch can use a highly optimized fused attention kernel that assumes all sequences have the same query length (1). A mixed batch needs a more general kernel that handles variable query lengths. The optimized kernels exist (FlashAttention, FlashInfer) but they're slightly slower than decode-only specialized versions.

Cost-per-token isn't uniform. A prefill token at iteration i attends to all of its own prior prompt tokens (linear in prompt length per token); a decode token attends to the full context (linear in context length). The kernel does both, so the total per-iteration work depends on the composition. A batch with one long prefill and many decodes runs slower than a batch with all decodes at the same token count.

6. Chunked Prefill

A naive scheduler would prefill an arriving request's entire prompt in a single iteration. For a 4K-token prompt that takes 4K tokens of forward-pass work, which at typical iteration costs is hundreds of milliseconds — and during that time, every other request in the batch waits.

Chunked prefill solves this by splitting long prefills across multiple iterations. The scheduler limits how many prompt tokens a single request can advance per step via `long_prefill_token_threshold` (typically 1024 or 2048). A 4K prompt becomes four iterations of 1K each, with decode requests for other sequences participating in each one.

The mechanics in code are simple: when `schedule()` picks up a prefill request from the waiting queue, it sets `new_tokens = min(remaining_prefill, long_prefill_token_threshold, token_budget_remaining)`. The request's `prefill_tokens_done` counter advances by that amount; the request stays in the running queue until `prefill_tokens_done >= prompt_tokens`, at which point it transitions into decode mode.

6.1 The latency trade-off

Chunked prefill is a clean latency-vs-throughput trade. With chunks, an in-flight decoder sees iterations of moderate cost — say 30 ms each — even when a long prefill is being processed concurrently. Without chunks, the same decoder would see one iteration of 300 ms followed by short iterations until the next prefill arrived. Total time spent prefilling the same prompt is identical (both sum to ~300 ms); what changes is who bears the latency.

Production deployments serving real-time interactive workloads (chat, code completion) always enable chunked prefill. Batch deployments where every request is similar size and waiting in line may disable it because the chunking overhead is wasted work.

7. KV-Pressure Preemption

The KV pool has finite capacity. If many requests run concurrently and each accumulates KV state as it decodes, eventually the pool fills. When `allocate_slots` fails — no blocks available — the scheduler must do something. It cannot just block the request; that would deadlock the entire engine. It cannot give the request fewer blocks than it needs to write; the next token would have nowhere to go.

The answer: **evict a running request** to free its blocks, returning the evicted request to the waiting queue so it can be re-admitted later. The two questions are: how do you pick the victim, and what do you do with its already-accumulated state?

7.1 V0 had two preemption strategies; V1 has one

V0 supported two recovery mechanisms:

Swap. Move the victim's KV blocks from GPU memory to CPU memory (slow but preserves the cache). When the request is later re-admitted, swap the blocks back. The cost is the swap-out and swap-in latency, but the request doesn't have to recompute prefill.

Recompute. Discard the victim's KV blocks entirely. When the request is later re-admitted, re-prefill its prompt plus the tokens it had already decoded. No swap latency but the prefill work is repeated.

V1 dropped swap. Recompute is the only V1 strategy. The reasoning involves several factors:

- **PCIe bandwidth bottleneck.** Swap moves data across PCIe, which is ~10× slower than HBM. A swap+swap-back cycle for a long context can take longer than recomputing the prefill, especially with chunked prefill amortizing the work.
- **Prefix caching reduces recompute cost.** When the victim is re-prefilled, the prefix-caching mechanism (Section 8) finds the previously-computed blocks if they still exist in the pool, skipping the redundant work. This makes recompute much cheaper than it sounds.
- **Simpler code path.** Swap requires managing CPU-side block pools, swap queues, and double-buffered transfers. Removing it simplifies the engine considerably.

7.2 Victim selection

When recompute fires, the scheduler picks the victim that costs the least to evict. The heuristic favors requests with the *least decode progress* — i.e., the lowest `decode_tokens_done` count. The reasoning: re-prefill cost scales with prompt length plus decoded tokens; evicting a request that has only decoded a few tokens means its re-prefill is essentially just its original prompt, which is cheap.

This produces a mild fairness bias: requests with very long outputs are protected from preemption once they're deep into decode, while newly-admitted decodes are vulnerable. There's a per-request

preemption cap (`max_preemptions`, default 5) to prevent the same victim from being evicted repeatedly.

7.3 The recompute mechanics

When a victim is evicted:

1. Its block list is returned to the free queue. The blocks are now available for the new request that triggered the eviction.

2. Its `prompt_tokens` is rewritten to `original_prompt_tokens + decode_tokens_done`. The next time it runs prefill, it must process this combined length.

3. Its `prefill_tokens_done` is reset to zero. Prefill starts over.

4. It is placed at the front of the waiting queue. On the next `schedule()` call, it gets first consideration before any new arrivals — soft fairness to avoid starvation.

5. Its `preemption_count` is incremented. If it reaches the limit, it is exempted from further evictions.

Note that the user-visible first_token_time is preserved — the user already received a token before the eviction. The cost of preemption appears as a long gap between successive tokens, which shows up in the time-between-tokens (TBT) metric.

8. Prefix Caching

Many real workloads share prompt prefixes. A multi-turn chat re-sends the entire conversation as context for each new turn. A few-shot prompt has a static template. A document-QA pipeline has a constant system prompt. In all of these, the first N tokens of every request are identical to a previous request's first N tokens.

Prefix caching exploits this. The KV state for those shared tokens has already been computed and stored in the cache; if the blocks are still allocated (because another request is using them, or because they haven't been evicted yet), the new request can simply point its block list at them rather than re-prefilling.

8.1 The hash table

Each block in the KV cache corresponds to a fixed window of `block_size` tokens (default 16). The cache key is a hash of: the tokens themselves, the position offset (because tokens at different positions have different KV state due to positional encoding), and the parent block's hash (so a hit requires matching the full prefix path, not just an isolated 16-token window).

When a request enters with prompt *P*, the scheduler hashes its first 16 tokens, checks if that block exists. If yes, hashes the next 16 with the previous block's hash, checks again, and so on. The longest matching prefix becomes the request's initial block list. Only the un-matched suffix needs new prefill work.

8.2 Block retention policy

When a request finishes, its blocks are released back to the free queue. With prefix caching enabled, they're not immediately recycled — they're added back to the head of the queue, but their hash entries remain in the cache. They get reclaimed only when actual demand requires it (i.e., another allocation needs them). This LRU-like behavior maximizes the chance that a future request with the same prefix gets a hit.

8.3 When it helps and when it doesn't

Prefix caching is enabled by default in V1. It produces near-zero overhead when there's no prefix sharing (the hash lookups are cheap; no allocations are skipped). When there's heavy sharing — typical for chat and few-shot workloads — it can reduce effective prefill work by 50-90%.

Workloads where it doesn't help: high-throughput batch inference on independent prompts (each prompt is unique), document summarization with no shared template, code generation with diverse inputs. In these cases the feature is harmless but provides no benefit.

9. Speculative Decoding (Brief)

Speculative decoding lets the engine produce more than one output token per forward pass. The idea: use a cheap mechanism to guess the next k tokens, then run the expensive model once to verify them. If the model accepts all k guesses, the request advances by k+1 tokens in one forward pass; if it rejects, only the verified tokens advance (with the rejection point becoming the new state).

V1 supports three speculation backends:

- **n-gram.** Matches a window of recent tokens against an n-gram lookup table built from common patterns in the prompt or generated text. Cheapest mechanism; works well for code or templated text where local patterns repeat.
- **EAGLE.** A small auxiliary model trained to predict the main model's next tokens. Substantially more accurate than n-gram; requires training and slightly more compute per draft.
- **Medusa.** Multiple heads added to the main model that predict k future tokens in parallel during a single forward pass. Tighter integration than EAGLE; requires model surgery.

For the scheduler, speculative decoding shows up as a per-request `spec_token_ids` list. When the request gets scheduled, `new_tokens` becomes `len(spec_token_ids) + 1` rather than 1. The KV cache must reserve room for all k+1 tokens up front, even though some may be rejected. The throughput gain when acceptance rates are high (often 60-90% for n-gram on suitable workloads) is substantial.

10. Performance Characteristics

10.1 The roofline knee

The fundamental cost shape: as concurrent batch size grows, iteration latency stays roughly flat up to a saturation point, then grows linearly above it.

The flat region is HBM-bandwidth-bound. Iteration time is dominated by loading the model weights, which is constant regardless of batch size. The compute portion (B × per-token-FLOPs) occupies a fraction of the time available.

The linear region is compute-bound. Above the saturation batch B_sat, the compute time exceeds the bandwidth time. Each additional sequence adds proportional work; iteration time grows linearly.

Theoretically, $B\_sat = F / BW$ where F is peak FLOPS and BW is HBM bandwidth. For an A100 80GB at bf16, F ≈ 312 TFLOPS, BW ≈ 2 TB/s, giving $B\_sat \approx 156$. In practice the measured value is lower (typically 16-64 for 7B-class models) because attention compute scales with context length, kernel launch overhead creates a per-iteration floor, and KV cache loading itself contributes traffic that's not captured by pure roofline.

10.2 Implications for scheduler tuning

Below B_sat, increasing concurrency is essentially free — more requests, same iteration time, more throughput. The scheduler can pack the batch aggressively.

Above B_sat, increasing concurrency costs latency for every concurrent request. Past a certain point, *total throughput drops* because iterations get slow enough that the higher batch can't compensate. There's a sweet spot near B_sat where throughput peaks.

Most production deployments configure `max_num_seqs` (the maximum running batch size) to be slightly above the measured B_sat so that the system can absorb arrival bursts without rejecting requests, but doesn't routinely operate deep in the compute-bound regime. The tuning question is how much slack to leave; this is workload-dependent.

10.3 Common metrics

The five metrics commonly reported by serving benchmarks:

- **TTFT (Time To First Token).** From request arrival to the first sampled output token. Dominated by prefill time, scheduler queuing, and any preemption events that delay re-admission.
- **ITL (Inter-Token Latency) / TPOT (Time Per Output Token).** Average time between successive output tokens after the first. Dominated by per-iteration cost.
- **TBT (Time Between Tokens).** Variance-sensitive version of ITL — measures the distribution rather than the mean. Spikes when preemption fires.
- **E2E (End-to-End Latency).** From arrival to last output token. Combines TTFT with output length × ITL.
- **Throughput.** Total output tokens per second, aggregated across all requests. The headline number for batch deployments.

Goodput — throughput satisfying SLOs — is increasingly used as the more honest metric. The vLLM benchmark tool (`vllm bench serve`) computes both.

11. A Worked Example

To make the pieces concrete, trace through what happens when three requests arrive over a few iterations.

Initial state: Free queue has 1000 blocks. Token budget 8192. max_batch 32. Block size 16. Running and waiting queues empty.

Time T=0, Request A arrives. Prompt = 100 tokens, max_output = 500 tokens. Processor tokenizes; A joins waiting queue.

T=0, step() phase 1: Running queue empty; skip.

T=0, step() phase 2: Waiting queue has A. With chunked prefill disabled (for simplicity), schedule 100 prefill tokens. allocate_slots reserves ⌈100/16⌉ = 7 blocks. Free queue now has 993 blocks. A moves to running queue. Budget remaining: 8092.

T=0, forward pass: Model processes 100 prefill tokens for A, samples 1 decode token, appends to A's KV cache and output. A's state: prefill done, 1 output token.

T=1, Request B arrives. Prompt = 50 tokens. Joins waiting queue.

T=1, step() phase 1: A is running, needs 1 new decode token. allocate_slots: A's last block (from prefill) has space for more tokens (room for 4 more in the 16-token block after 100-tokens-prefilled). No new block needed. Budget remaining: 8191.

T=1, step() phase 2: B waiting. Schedule 50 prefill tokens. allocate_slots reserves ⌈50/16⌉ = 4 blocks. Free queue: 989 blocks. B moves to running. Budget remaining: 8141.

T=1, forward pass: Mixed batch — A's 1 decode token + B's 50 prefill tokens. Total tokens this forward = 51. Both have new K/V written. A gets its 2nd output token; B gets its 1st.

This cycle repeats. Over the next many iterations, A and B accumulate decode tokens. Their block lists grow as each new block fills up. New requests arrive; some pass through quickly when capacity is loose, others queue when it's tight.

Sometime later: Suppose 40 requests have accumulated, all decoding, and the free queue is at 5 blocks. A new request C arrives with a 200-token prompt. The scheduler tries to admit C in phase 2; allocate_slots needs 13 blocks but only 5 are available. allocate_slots fails. The scheduler picks the lowest-progress running request — say a request F that has only decoded 3 tokens. F's blocks are released back to the queue (say 8 blocks), giving 13 free. F is requeued at the front of waiting; its prompt_tokens is set to original_prompt + 3. C is admitted, its 13 blocks allocated, prefill begins.

The user owning F will see a gap between their 3rd and 4th tokens equal to the time from preemption to when F is later re-admitted and re-prefilled. The user owning C will see prompt processing without delay.

12. Closing Notes

vLLM's scheduler is a careful balance between simple, predictable behavior and aggressive performance optimizations. The block-based KV cache, the FIFO admission order, the decode-before-prefill priority, and the recompute-only preemption strategy are all designed to be robust under adversarial workloads while still achieving near-peak throughput when load is well-behaved.

The areas of active development in V1 are largely about extending what we've described:

- **Disaggregated prefill/decode.** Running prefill on one set of GPUs and decode on another, with KV cache transfer between them. Decouples the throughput-bound prefill (compute-heavy) from the latency-bound decode (memory-heavy).
- **Multi-LoRA.** Serving many fine-tuned adapter variants simultaneously without recompiling the model. Touches scheduling because each LoRA's adapter weights need to be loaded into HBM, and the scheduler may want to group requests by LoRA to reduce cache thrashing.
- **Heterogeneous quantization.** Different requests may use different quantization levels (FP16 for accuracy-sensitive workloads, INT8 for throughput-sensitive ones). The scheduler considers this in batching decisions.

None of these change the core decisions we've described — they extend them. Understanding the scheduler-and-KV-manager pair as documented here is sufficient to reason about the engine's behavior under any extension.

For the canonical reference, the V1 source is at `github.com/vllm-project/vllm`, with the scheduler in `vllm/v1/core/sched/scheduler.py` and the KV cache manager in `vllm/v1/core/kv_cache_manager.py`. Aleksa Gordic's article "Inside vLLM: Anatomy of a High-Throughput LLM Inference System" (aleksagordic.com/blog/vllm) is a thorough engineering tour that goes substantially deeper than these notes.